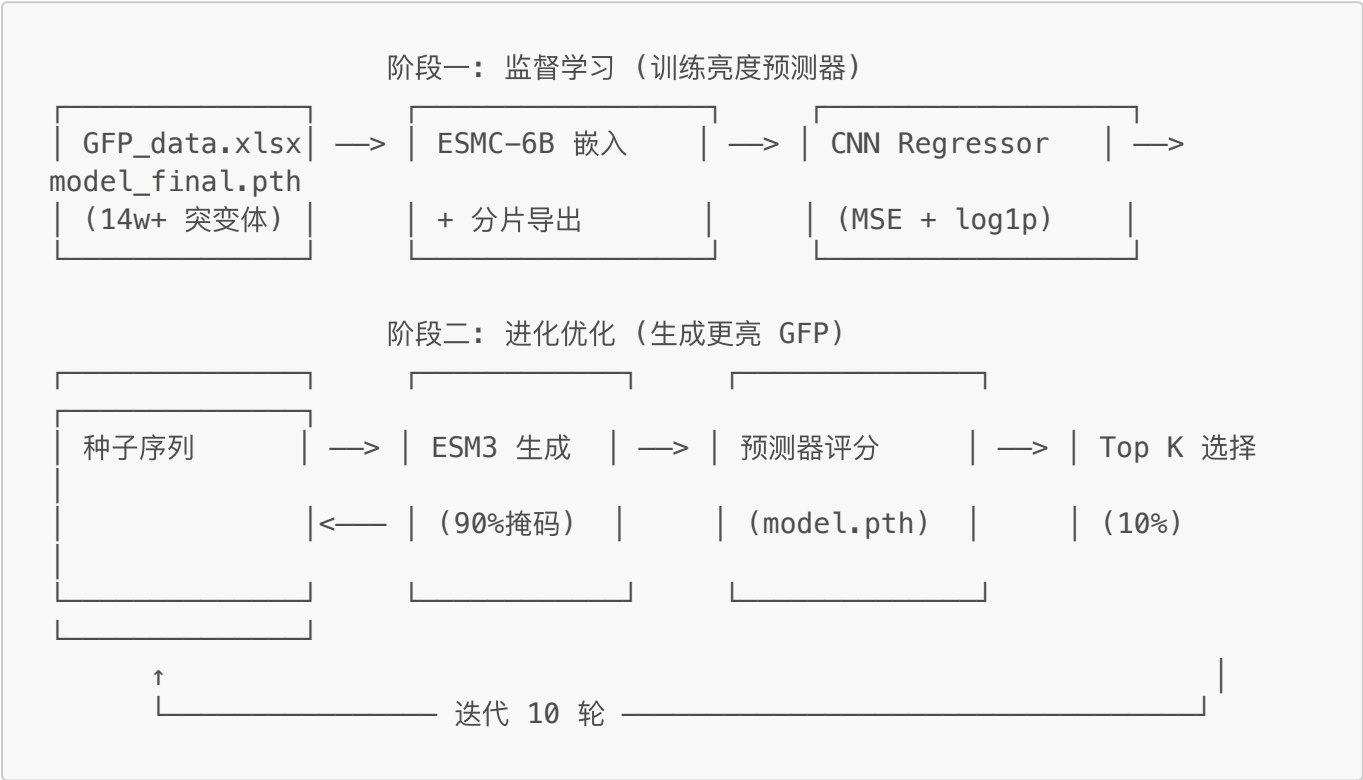


2026ProteinDesign — GFP 荧光亮度预测与进化优化

基于 ESM 蛋白语言模型嵌入 + CNN 回归预测 GFP 荧光亮度，并结合 ESM3 掩码语言模型进行进化优化，设计更高亮度的 GFP 蛋白变体。合成生物学竞赛项目。

整体流程



环境配置

```
conda create -n esm python=3.12
conda activate esm
pip install -r requirements.txt
```

依赖项

包	用途
torch	深度学习框架
numpy	数值计算
scikit-learn	数据集划分与 R² 评估
transformers	HuggingFace 模型加载 (ESMC-6B)
pandas / openpyxl	读取 Excel 数据
tqdm	进度条

数据准备

将原始 `GFP_data.xlsx` 转换为训练数据集（含 ESM 嵌入）：

```
python utils/export_dataset_json.py
```

14 万条数据采用分片导出，避免大数据量卡死。每 1000 条记录为一个分片，ESM 嵌入在存储前做平均池化（ $250 \times 2560 \rightarrow 2560$ ），大幅降低存储占用（ $\sim 341 \text{ GB} \rightarrow \sim 1.37 \text{ GB}$ ）。

输出文件：

- `gfp_dataset_shard_000.json ... _shard_NNN.json` — 各分片序列与亮度标注
- `gfp_dataset_shard_000_embeddings.npy ... _shard_NNN_embeddings.npy` — 各分片池化嵌入 $[1000, 2560]$
- `gfp_dataset_shards.json` — 分片清单（总记录数、分片数、路径映射）

训练

```
python model/train.py
```

训练配置（`model/train.py` 内）：

- 模型: `BrightnessRegressor` — 3 层 Conv1d + 2 层 FC
- 输入: 平均池化后的 2560 维 ESM 嵌入（ $[N, 2560]$ ）
- 损失: MSE（对 `log1p` 变换后的亮度值）
- 优化器: Adam, `lr=1e-4`, ReduceLROnPlateau 调度
- 数据划分: 80/10/10 (train/val/test), `seed=114514`
- 早停: `patience=50`, 梯度裁剪 `max_norm=1.0`
- 分片数据集通过 `np.load(mmap_mode='r')` 延迟加载，内存友好
- 输出: `model_final.pth`, `evaluation_results.json`, `training_logs.json`

评估

对单条序列进行亮度预测：

```
python model/eval.py
```

输出野生型 GFP 的预测亮度值。

在代码中使用：

```
from model.eval import EVAL

evaluator = EVAL("model_final.pth")
```

```
brightness = evaluator.predict("MSKGEELFTGVV...")
print(f"预测亮度: {brightness:.4f}")
```

进化优化管线

运行完整的生成-评分-选择迭代管线：

```
python main.py
```

默认配置（`main.py` 内 `CONFIG` 字典）：

参数	默认值	说明
初始序列	<code>_____AAB_____</code>	种子掩码序列
每父代生成数	200	每轮每个父代生成的子序列数
Top K 比例	0.1	每轮保留比例
最大父代数	20	每轮保留上限
掩码率	0.9	子序列中随机掩码的残基比例
温度	1.0	ESM3 生成温度（实际加 $\pm 20\%$ 随机扰动）
生成轮数	10	迭代轮数
序列长度	225-250	过滤有效序列的长度范围

每轮结果保存到 `pipeline_results.json`，最终输出最佳序列与亮度。

一键运行

```
bash run.sh
```

依次执行：环境激活 → 依赖安装 → 数据集导出 → 训练（日志保存到 `logs/` 目录）。

项目结构

.

├── main.py

├── run.sh

├── requirements.txt

├── GFP_data.xlsx

├── model/

│ ├── BrightnessRegressor.py

│ ├── train.py

│ └── eval.py

进化优化主管线

一键运行脚本

Python 依赖

原始突变体数据

CNN 回归模型定义

训练脚本

推理封装类

```
├── esm_utils/
│   ├── esmc_embedding.py          # ESMC-6B 序列嵌入
│   ├── esm3_generate.py          # ESM3 掩码序列生成
│   ├── esmfold2_generate.py      # ESMFold2 结构预测（独立工具）
│   └── wt_extract.py             # 野生型嵌入提取
├── utils/
│   ├── export_dataset_json.py    # Excel → 分片 JSON + 嵌入（池化）
│   └── convert_gfp_data.py       # 突变解析与序列构建
└── checkpoints/                 # 训练检查点
```

技术细节

模型架构

```
Input: [batch, 2560] (平均池化后的 ESM 嵌入)
  → unsqueeze → [batch, 1, 2560]
  → Conv1d(1→32, k=7) → BN → GELU → MaxPool1d(2) → [batch, 32, 1280]
  → Conv1d(32→64, k=5) → BN → GELU → MaxPool1d(2) → [batch, 64, 640]
  → Conv1d(64→128, k=3) → BN → GELU → AdaptiveAvgPool1d(1) → [batch, 128]
  → Linear(128→64) → GELU → Dropout(0.1) → Linear(64→1)
Output: [batch] (预测亮度, log1p 空间)
```

数据说明

- 训练数据来自 5 种野生型 GFP (sfGFP, avGFP, amacGFP, cgreGFP, ppluGFP) 及其突变体, 共 14w+ 条
- 亮度值经过 **log1p** 变换后作为回归目标, 取值约 1.3 ~ 3.8
- 使用 ESMC-6B 模型提取每残基 2560 维嵌入, 在存储前做平均池化压缩为单向量
- 序列过滤条件: 长度 225~250 (超出范围的直接丢弃)